

课程笔记

[LLM Agents SP25] Open Training Recipes: LLM Reasoning

基于 Hanna Hajishirzi 授课内容整理

2026-04-02



视频作者/频道: Berkeley RDI

发布日期: 2025-03-03

视频时长: 1:20:53

视频链接: <https://www.youtube.com/watch?v=cMiu3A7YBks>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 讲座定位：为什么“训练配方”比单点技巧更重要	3
1.1 问题定义：Reasoning 不是单一能力，而是训练链路的系统产物	3
1.2 从“模型发布”到“科学发布”	3
1.3 本章小结	4
2 三阶段框架：Pre-training, Post-training, Inference-time	4
2.1 统一视角：能力形成的三层杠杆	4
2.2 工程化分解：每层解决哪类瓶颈	5
2.3 本章小结	5
3 Pre-training：OLMo 2 的可复现经验	6
3.1 目标：在可控成本内做出高竞争力底座	6
3.2 预训练里真正影响推理底座的因素	6
3.3 本章小结	7
4 Post-training I：SFT 与数据工程	7
4.1 SFT 的角色：把潜在能力变成可调用行为	7
4.2 人类数据与合成数据的混合策略	8
4.3 本章小结	8
5 Post-training II：偏好优化（DPO/PPO）	8
5.1 从监督信号到偏好信号	8
5.2 DPO 与 PPO 的工程权衡	9
5.3 本章小结	9
6 Post-training III：RLVR 与可验证奖励	10
6.1 为什么 RLVR 成为推理提升的关键转折	10
6.2 神经奖励模型与规则奖励的比较	10
6.3 从 DeepSeek-R1 经验到开放社区可复现实践	11
6.4 本章小结	11
7 Inference-time：Test-time Scaling 的工程价值	11
7.1 核心思想：把额外计算放在最需要的样本上	11
7.2 部署侧权衡：准确率、延迟与成本	12
7.3 本章小结	13
8 端到端配方：从 OLMo 到 Tulu 的组合逻辑	13
8.1 组合原则：先稳底座，再做对齐，最后做预算优化	13
8.2 评估体系：不要只看一个 benchmark	14
8.3 本章小结	14

9 风险与研究议程：开放训练下一步该做什么	14
9.1 当前最关键的三类研究缺口	14
9.2 给研究者和产品团队的两条建议	15
9.3 本章小结	15
10 实施手册：把训练配方变成团队可执行流程	15
10.1 四周迭代节奏示例	15
10.2 实验记录模板：最小可追溯字段	16
10.3 从研究到上线：灰度发布检查单	16
10.4 本章小结	16
11 案例拆解：一次 Reasoning 升级实验如何设计	16
11.1 实验目标与约束	16
11.2 分阶段实验矩阵	17
11.3 错误分析优先级	17
11.4 把案例映射回本讲主线	17
11.5 本章小结	18
12 总结与延伸	18
12.1 全讲总结表	18
12.2 核心结论	18
12.3 进一步阅读	18
12.4 延伸问题	19

1 讲座定位：为什么“训练配方”比单点技巧更重要

1.1 问题定义：Reasoning 不是单一能力，而是训练链路的系统产物

本讲最核心的价值不是再讲一个新算法，而是把“reasoning 能力”拆回工程现实：它由预训练分布、后训练数据、优化目标、验证机制、推理时预算共同决定。也就是说，推理能力不是某个 magical trick 的直接结果，而是一个端到端系统设计问题。

讲者的核心立场

“Open scientific research is why we are here.” 讲者把过去几年 LLM 进展归因于开放科研生态，并强调当生态变得封闭，研究可复现性、可比较性和可纠错性都会下降。

为什么这一讲要强调“recipes”

模型能力讨论常被简化为参数规模与算力，但工业可复现的“recipe”其实包含更多隐含变量：

- 数据采样与混合策略；
- 训练阶段切分（pre-train / post-train / inference-time）；
- 评估方式与去污染策略；
- 预算约束下的最优折中。

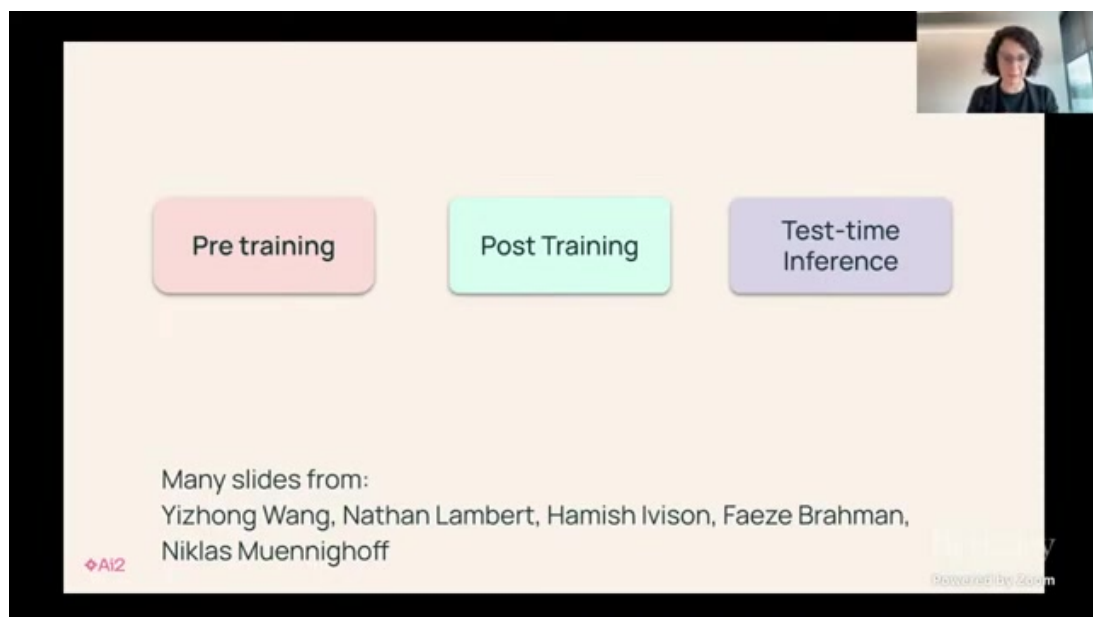


图 1: 视频约 00:03:10：讲者明确把“开放研究”作为训练配方方法论的前提

1.2 从“模型发布”到“科学发布”

讲者把开放模型分成三个层级：仅开放权重、开放训练代码、开放数据与评估流程。真正支持科学研究的是第三层，因为只有同时开放数据处理、训练脚本和评估基准，研究社区才能判断改进来自哪里。

只开权重不等于可复现

很多工作只发布 checkpoint，导致研究者无法验证提升是来自：

- 数据质量改善；
- 训练超参数微调；
- 优化器与调度修改；
- 评估污染或基准差异。

当这些变量不可见时，所谓“SOTA”对科学社区的价值会明显下降。

1.3 本章小结

本章给出整场讲座的方法论基线：推理能力要从“完整训练配方”理解，开放与可复现是该配方可累积迭代的必要条件。

2 三阶段框架：Pre-training, Post-training, Inference-time

2.1 统一视角：能力形成的三层杠杆

讲者把训练体系拆成三层杠杆：预训练决定底座容量与知识覆盖，后训练决定行为风格与任务偏好，推理时扩展决定单位请求可投入的搜索计算量。三者互补且存在明显的边际收益差异。

三阶段不是替代关系，而是分工关系

- **Pre-training**：提供广覆盖表征能力与知识压缩；
- **Post-training**：把能力定向到可用行为（对话、指令遵循、偏好一致）；
- **Inference-time**：在部署时动态投入额外计算，提高难任务成功率。

Getting Ingredients to Start With

Successful adaptation starts with:

1. Meaningful **evaluations** for targeted skills
2. **Prompts** of representative queries for said skills
3. Check for Licenses
4. Decontamination

AI2

Category	Prompt Dataset	Count	# Prompts used in SFT	# Prompts used in DPO
General	Tulu 3 Hardcoded ¹	24	240	-
	OpenAssistant ^{1,2,1}	88,838	7,132	7,132
	No Robots	9,500	9,500	9,500
	WildChat (GPT-4 subset) ²	241,307	100,000	100,000
	UltraFeedback ^{2,2}	41,635	-	41,635
Knowledge	FLAN v2 ^{1,2,1}	89,982	89,982	12,141
Recall	SciRIFF ³	35,357	10,000	17,590
	TableGPT ¹	13,222	5,000	6,049
Math	Tulu 3 Persona MATH	149,960	149,960	-
Reasoning	Tulu 3 Persona GSM	49,980	49,980	-
	Tulu 3 Persona Algebra	20,000	20,000	-
	OpenMathInstruct 2 ¹	21,972,791	50,000	26,356
	NamimaMath-TIR ⁴	64,312	64,312	8,677
Coding	Tulu 3 Persona Python	34,999	34,999	-
	Evol CodeAlpaca ⁴	107,276	107,276	14,200
Safety	Tulu 3 CoCoNot	10,983	10,983	10,983
& Non-Compliance	Tulu 3 WildJailbreak ^{4,1}	50,000	50,000	26,356
	Tulu 3 WildGuardMix ^{4,1}	50,000	50,000	26,356
Multilingual	Aya ¹	202,285	100,000	32,210
Precise IF	Tulu 3 Persona IF	29,980	29,980	19,850
	Tulu 3 IF-augmented	65,530	-	65,530
Total		23,327,961	939,344	

图 2: 视频约 00:11:40: 讲者展示从预训练到后训练再到测试时扩展的整体栈

2.2 工程化分解：每层解决哪类瓶颈

阶段	主要目标	典型技术	常见瓶颈
Pre-training	学表示、压缩知识、打基础能力	数据配比、优化器、学习率调度、架构细节	数据质量上限、算力成本高
Post-training	对齐行为、增强特定能力	SFT、DPO/PPO、RLVR	数据分布冲突、reward 偏差
Inference-time	提高难题成功率与鲁棒性	Best-of-N、自一致性、检索迭代	延迟与成本上升、误判放大

表 1: 三阶段训练框架及其瓶颈分工

为什么课程里要反复强调“组合”

单一阶段很难独立解决推理问题。比如只做后训练会受限干底座能力，只做推理时扩展会受限干候选质量。因此更现实的路径是按预算做“可叠加组合”，而不是寻找万能算法。

2.3 本章小结

三阶段框架提供了分析推理模型的最小完备视角：先看底座，再看对齐，最后看部署时计算分配。

3 Pre-training: OLMo 2 的可复现经验

3.1 目标：在可控成本内做出高竞争力底座

讲者用 OLMo 2 讨论了一个关键现实：研究型团队并不总有 frontier 级算力，但依然可以通过更好的数据治理与训练细节，在 7B/13B 规模做出强基线。

OLMo 2 的价值在于“可解释改进”

相较只公布结果曲线，OLMo 系列更强调公开训练链路，使社区可以系统回答：

- 哪些训练细节真正贡献了性能；
- 哪些改动只在特定基准有效；
- 在不同预算下哪组配置更稳健。

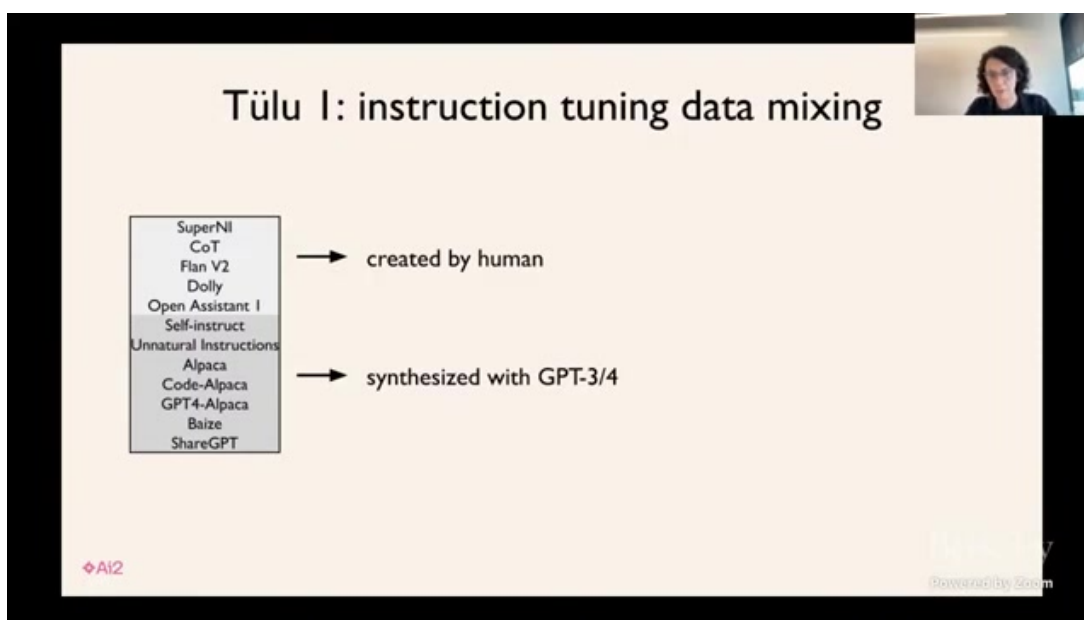


图 3: 视频约 00:18:30: OLMo 训练曲线与规模比较，强调在预算受限下的效率优化

3.2 预训练里真正影响推理底座的因素

比“参数规模”更容易被忽略的四类变量

1. 数据去重、污染控制与质量分层；
2. 领域采样比例（代码、数学、知识文本、对话）；
3. 学习率与 warmup/cooldown 调度；
4. tokenizer 与上下文窗口策略。

讲者强调，推理能力在预训练阶段已经被部分决定，后训练只能在已有能力边界内做重排和强化。这解释了为何很多后训练技巧在不同底座上效果差异很大。

预训练指标与下游推理能力并非一一对应

较低的语言建模 loss 不必然带来更强链式推理。若训练语料中的推理结构信号不足，模型可能擅长流畅续写却不擅长多步约束推断。

3.3 本章小结

OLMo 2 的启示是：在研究场景中，公开且可解释的预训练配方本身就是成果，它决定了后续所有 reasoning 增强的上限与可重复性。

4 Post-training I: SFT 与数据工程

4.1 SFT 的角色：把潜在能力变成可调用行为

SFT 不是“补课”，而是建立交互协议。它让底座模型知道任务边界、输出风格和回答结构，直接影响后续偏好优化与 RL 的稳定性。

讲者给出的实务判断

若 SFT 数据结构混乱、指令覆盖窄、质量参差不齐，后续 DPO/PPO 与 RLVR 会更容易出现 reward hacking、模式坍缩或能力偏科。

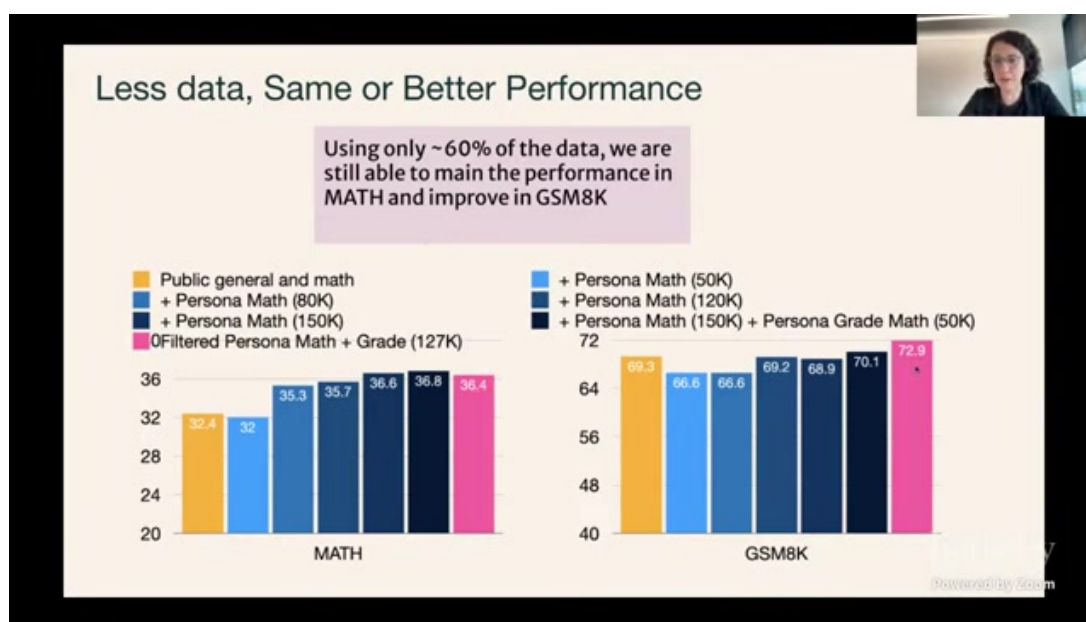


图 4: 视频约 00:29:30: SFT 阶段的数据来源与清洗策略，强调人类数据与合成数据的协同

4.2 人类数据与合成数据的混合策略

混合的目标不是“谁替代谁”，而是“谁补谁”

- 人类标注数据：高质量、风格真实，但规模小且成本高；
- 合成数据：规模大、覆盖广，但质量波动与偏差风险更高；
- 最优策略通常是高质量人类样本定锚，合成样本扩展覆盖。

讲者多次提到数据去污染（decontamination）与许可证治理，这对开放社区尤为关键。因为在 benchmark 污染存在时，训练改进会被误读为算法进步。

SFT 最常见的误区

只看总量不看分布。堆更多 instruction 数据若破坏任务平衡，可能导致推理能力上升但事实性、格式遵循或多语言能力下降。

4.3 本章小结

SFT 阶段的核心是数据工程而不是单一算法。可复现的数据治理流程，是后续偏好优化与强化学习稳定工作的前置条件。

5 Post-training II: 偏好优化 (DPO/PPO)

5.1 从监督信号到偏好信号

在 SFT 后，模型已具备基本任务执行能力，但答案风格、风险偏好、简洁性与有用性仍需通过偏好学习进一步塑形。讲者围绕 DPO 与 PPO 的实践差异给出了大量工程观察。

为什么偏好优化对 reasoning 有效

Reasoning 任务常存在多个可行解，偏好优化能够引导模型朝更可解释、步骤更完整、错误更可纠正的输出分布移动，而不是仅追求单点监督标签匹配。

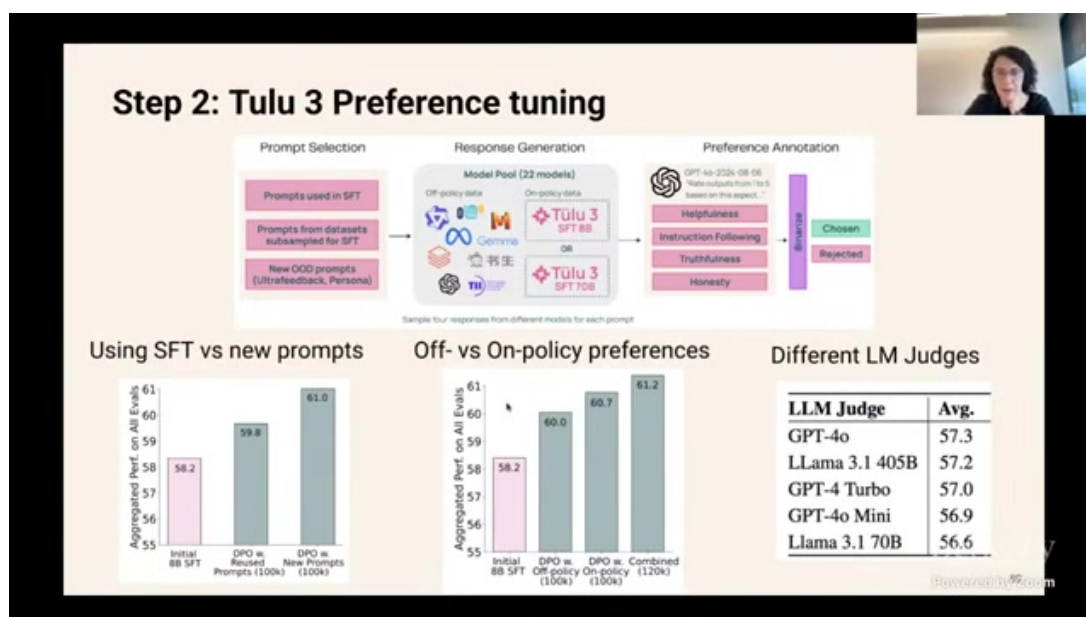


图 5: 视频约 00:48:30: DPO/PPO 对比及训练稳定性讨论

5.2 DPO 与 PPO 的工程权衡

方法	优势	风险	适用阶段
DPO	实现相对简单，成本较低，训练链路短	偏好样本质量敏感，长度偏置需处理	大规模快速迭代
PPO/在线 RL	可利用在线反馈持续优化，探索更强	不稳定、调参复杂、成本高	关键能力冲刺阶段

表 2: DPO 与 PPO 的实务差异

课程里反复出现的一个主题

算法替换往往不是决定性因素，**奖励信号质量与训练分布覆盖**通常更关键。很多“算法收益”实际上来自数据与目标函数修正。

偏好学习的两类隐性退化

- **过度迎合:** 模型学会“看起来像好答案”，但缺少真实推理深度；
- **长度捷径:** 模型通过拉长或缩短输出触发 reward 偏好，而非提升正确性。

5.3 本章小结

偏好优化是后训练中的行为控制层。DPO/PPO 不是二选一，而是要结合数据质量、预算和稳定性目标进行分阶段部署。

6 Post-training III: RLVR 与可验证奖励

6.1 为什么 RLVR 成为推理提升的关键转折

讲者将 RLVR (Reinforcement Learning with Verifiable Rewards) 定义为后训练中的关键拐点：当任务可以自动验证正确与否时，模型可以从大量自生成样本中持续改进，而不再完全依赖昂贵的人类偏好标注。

RLVR 的最小闭环

1. 生成候选答案；
2. 用规则或程序验证器给出可验证奖励；
3. 用 RL 更新策略，提高高奖励样本概率。

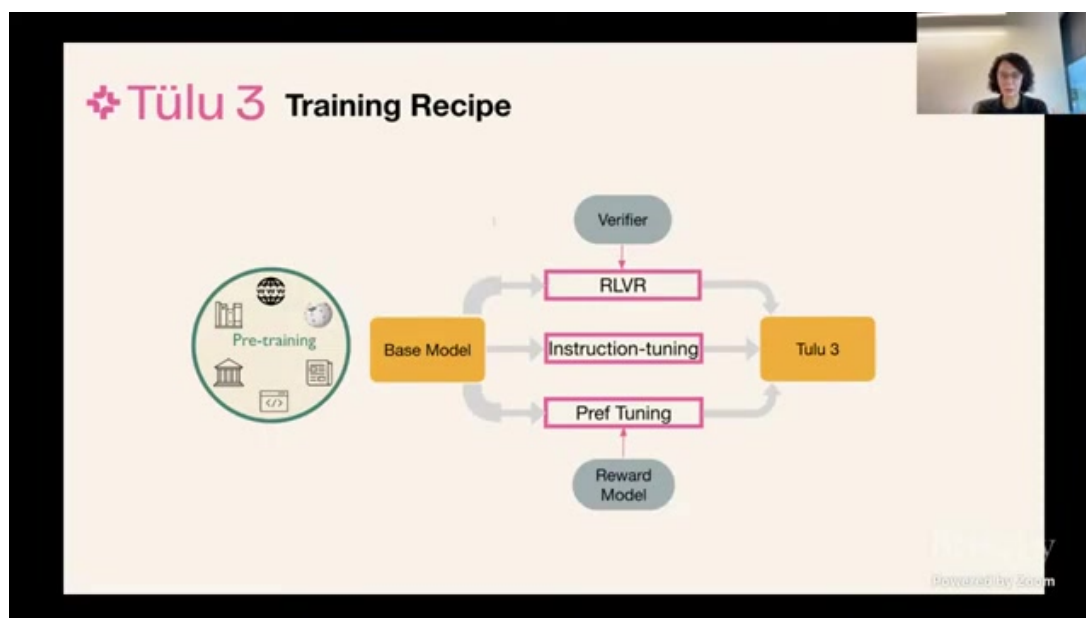


图 6: 视频约 01:03:20: 从神经 reward model 转向可验证奖励，降低标注依赖

6.2 神经奖励模型与规则奖励的比较

为何讲者强调“可验证”

神经奖励模型灵活但存在偏差与不可解释性；规则奖励更窄但更可信。对数学、代码、符号推理等任务，规则奖励能显著降低 reward hacking 风险。

奖励类型	优点	缺点	典型任务
神经奖励模型	表达能力强, 可覆盖复杂主观目标	易偏置、可解释性弱、成本高	对话偏好、风格一致性
规则/可验证奖励	稳定、可审计、可自动化规模训练	覆盖范围受限, 设计成本高	数学、代码、形式化约束

表 3: 两类奖励机制对比

RLVR 的边界条件

RLVR 并不自动提升所有任务。若任务缺少可靠验证器，或者验证器与真实目标错位，训练会把模型推向“高分低质”的错误方向。

6.3 从 DeepSeek-R1 经验到开放社区可复现实践

讲者引用了 R1 风格路线的启发：当验证器便宜、样本可规模生成时，RL 能迅速放大推理能力。对开放社区而言，关键在于把验证器、训练脚本和评估流程一起开源，否则很难验证结论可迁移性。

6.4 本章小结

RLVR 的真正价值在于把“推理提升”从高度人工依赖转为“可程序化迭代”。前提是验证器质量可靠且与真实目标一致。

7 Inference-time: Test-time Scaling 的工程价值

7.1 核心思想：把额外计算放在最需要的样本上

讲者把测试时扩展看作第三种杠杆：不改参数，只在推理时增加采样、校验、检索或反思步骤，让难题获得更多计算预算。这种策略通常对长链推理最有效。

Test-time Scaling 的三类常用策略

- **Best-of-N / self-consistency**: 多样采样后通过规则或模型选择；
- **检索增强迭代**: 在推理中间引入外部证据，动态修正；
- **过程约束搜索**: 用过程评分或约束过滤中间轨迹。

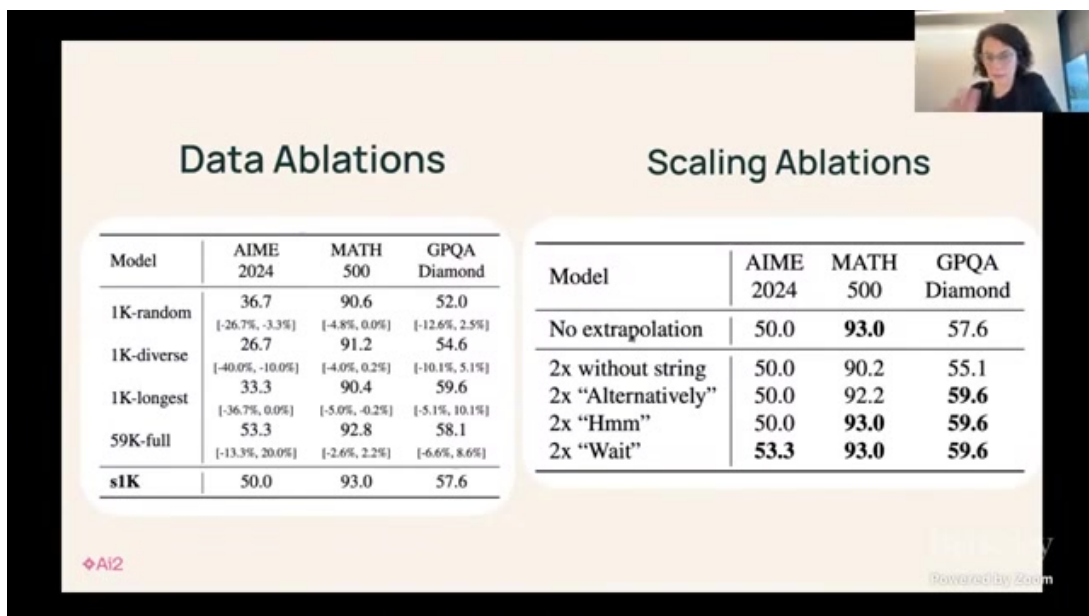


图 7: 视频约 01:12:40: 测试时扩展在推理任务上的收益与成本权衡

7.2 部署侧权衡：准确率、延迟与成本

策略	准确率变化	延迟/成本变化	适合场景
单次解码	基线	最低	在线高并发场景
Best-of-N	通常明显提升	随 N 线性增加	高价值低频请求
自一致性 + 验证	对可验证任务收益大	中高	数学/代码/结构化推理
检索迭代 (Self-RAG)	事实性与可追溯性提升	依赖检索与重排成本	知识密集问答

表 4: 测试时扩展的部署权衡

测试时扩展不是免费午餐

若基础模型候选质量差，多采样只会放大错误；若验证器不可靠，选择过程会将噪声当信号。因此 test-time scaling 的上限受底座质量与校验机制双重约束。

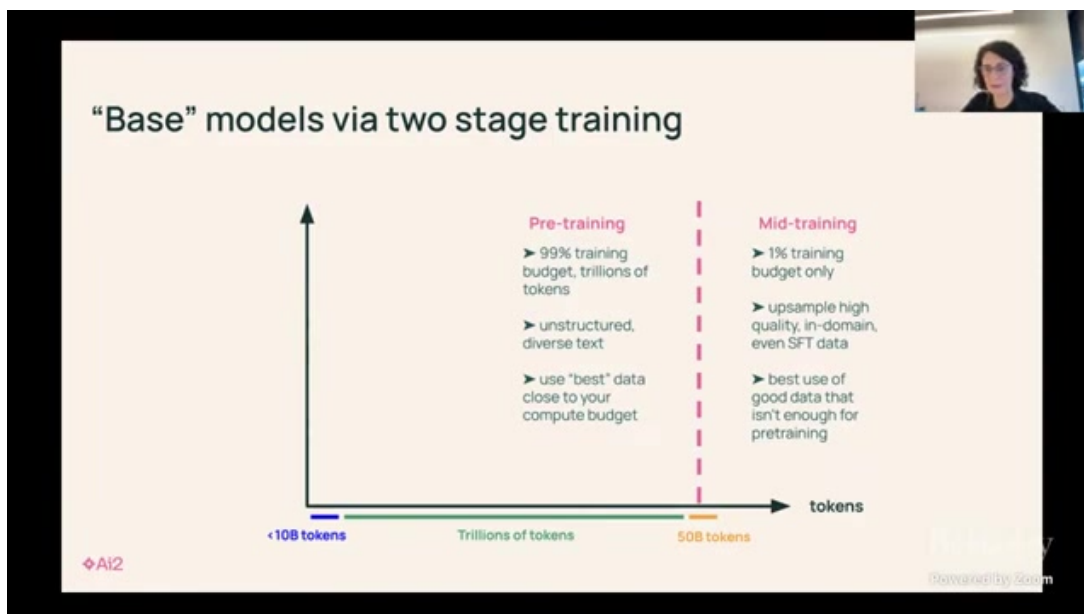


图 8: 视频约 01:17:20: Self-RAG 风格方法体现“推理-检索-校验”的闭环思想

7.3 本章小结

测试时扩展是部署侧的精细化预算分配工具，适合高难度、高价值请求，但必须与可靠验证机制配套使用。

8 端到端配方：从 OLMo 到 Tulu 的组合逻辑

8.1 组合原则：先稳底座，再做对齐，最后做预算优化

这场讲座给出的真正实践路线不是“先选算法”，而是“先选组合顺序”。一个可执行的顺序是：

1. 预训练阶段先建立稳定底座；
2. SFT 把任务协议和输出行为校正到可控范围；
3. DPO/PPO 做偏好细调；
4. RLVR 在可验证子域冲刺推理；
5. 测试时扩展服务于高价值长尾请求。

为什么这一路线对开放社区友好

每一层都可以独立实验、独立评估、独立开源，降低了研究协作门槛，也让复现实验失败时更容易定位问题所在。

8.2 评估体系：不要只看一个 benchmark

讲座中的评估观

Reasoning 提升必须跨多个维度共同验证：

- 数学/代码等可验证任务；
- 知识问答与事实性；
- 长上下文稳定性；
- 多语言与安全性约束；
- 成本与延迟指标。

指标成功不等于系统成功

如果评估不覆盖部署约束（延迟、预算、鲁棒性），模型即使 benchmark 提升，也可能在真实产品环境不可用。

8.3 本章小结

端到端配方的核心是顺序和组合，而不是单点最优。成功系统通常是多阶段“次优但互补”的结果。

9 风险与研究议程：开放训练下一步该做什么

9.1 当前最关键的三类研究缺口

缺口	现状问题	下一步方向
验证器生态	可验证任务覆盖仍窄，跨领域迁移弱	构建可组合 verifier 库与任务分层标准
数据治理	合成数据质量波动，污染难以统一审计	统一去污染协议与开源数据审计工具链
成本可控性	测试时扩展收益与成本预测不稳定	建立“质量-延迟-成本”联合调度策略

表 5: 讲座内容可落地的三类研究议程

对工程团队的直接启发

将“训练配方”文档化与版本化，与代码同等管理。每次能力变化都应可追溯到数据、目标函数、优化器、推理策略中的具体改动。

9.2 给研究者和产品团队的两条建议

建议 1：优先建设可复现实验底座

即便短期不追最高分，也要先把训练、评估、回归测试跑通。缺乏可复现基础时，任何改进都可能不可持续。

建议 2：把推理预算当作产品参数

推理时扩展不是算法附属，而是产品策略。要按请求价值分层分配预算，而不是对所有请求使用同一解码策略。

开放并不意味着低标准

开放训练路线同样需要严格的数据合规、评估去污染和安全审查。否则开放会变成复制噪声，而不是复制进步。

9.3 本章小结

下一阶段开放训练的胜负手，不是某个单一新算法，而是验证器生态、数据治理和推理预算调度这三件基础设施。

10 实施手册：把训练配方变成团队可执行流程

10.1 四周迭代节奏示例

为了让本讲方法真正落地，最实用的做法是把训练配方变成固定节奏，而不是临时实验。下面给出一个四周节奏模板，可用于中等规模研究团队：

1. 第 1 周：冻结目标任务与评估协议，完成数据去污染与基线跑通；
2. 第 2 周：SFT 数据混合实验，确认行为稳定区间；
3. 第 3 周：DPO/PPO 小规模扫描，验证偏好增益和副作用；
4. 第 4 周：RLVR + test-time scaling 联合实验，输出部署成本曲线。

让“实验节奏”比“灵感”更重要

推理能力提升往往不是一次跳变，而是连续小改动的累积。将每周目标、评估指标和回归测试固定化，才能避免团队反复在同一坑里循环。

10.2 实验记录模板：最小可追溯字段

字段	记录内容	缺失后果
数据版本	样本来源、去重规则、配比、许可状态	无法判断增益是算法还是数据漂移
训练配置	优化器、学习率、batch、训练步数、seed	结果不可复现，难以回归
奖励定义	偏好规则/验证器版本/阈值	难以排查 reward hacking
评估协议	基准集合、去污染规则、置信区间	分数不可比较，决策失真
部署预算	平均延迟、P95、token 成本、失败重试率	无法判断是否可上线

表 6: 将训练配方制度化的最小记录字段

工程上最容易被忽视的一点

很多团队只记录“最好成绩”，不记录“失败实验”。但在 reasoning 训练中，失败轨迹往往比成功轨迹更能揭示数据污染、奖励偏置或采样策略问题。

10.3 从研究到上线：灰度发布检查单

上线前必须完成的四项检查

- **正确性灰度**：在高价值任务上先限制流量，验证真实通过率；
- **成本灰度**：对比单次解码与 test-time scaling 的收益/成本；
- **稳定性灰度**：观察长链输出是否出现格式漂移与重复推理；
- **安全灰度**：对高风险提示词与越权请求做红队回归。

10.4 本章小结

训练配方案能否产生长期价值，取决于是否被制度化为团队流程。固定节奏、完整记录和灰度检查，是把研究成果转成稳定产品能力的关键。

11 案例拆解：一次 Reasoning 升级实验如何设计

11.1 实验目标与约束

假设我们要把一个 7B 级模型在数学与代码推理任务上提升 5-10 个点，同时不明显损伤通用对话能力。预算约束是“训练可控、推理可上线”。这类问题正好对应本讲提出的组合路线。

目标设定方式

目标应同时包含：

- **能力目标**：可验证任务准确率提升；
- **副作用上限**：通用任务下降不超过阈值；
- **部署目标**：成本和延迟在可接受区间。

缺一项都会让优化方向偏离真实需求。

11.2 分阶段实验矩阵

阶段	变量	观测指标	停止条件
SFT	人类/合成数据配比、长度分布	指令遵循、格式稳定性、基础正确率	副作用超过阈值即回滚
DPO/PPO	偏好损失权重、采样温度	有用性、冗长度、拒答率	偏好收益边际下降
RLVR	验证器规则、采样轮数、更新步数	数学/代码可验证通过率	出现奖励捷径信号
Inference-time	N 值、重排策略、检索轮次	通过率、P95 延迟、单位请求成本	成本超预算

表 7: Reasoning 升级实验的分阶段矩阵

11.3 错误分析优先级

最有价值的三类错误样本

1. **高置信错误**：模型给出自信但错误的长链推理；
2. **验证器误判**：答案正确但被规则误杀，或相反；
3. **预算不经济样本**：需要大量采样才提升极少准确率。

优先修复这些样本，通常能同时提升质量与成本效率。

不要用平均分掩盖结构性失败

平均分提升可能掩盖关键任务退化。真实系统更需要看“任务分桶表现”：例如多步数学、长代码修复、检索依赖问答分别变化如何。

11.4 把案例映射回本讲主线

这个案例表明，本讲的三阶段框架和开放配方思想可以直接转化为工程决策：

- 预训练决定底座上限；

- 后训练决定行为方向；
- 推理时扩展决定部署侧收益曲线；
- 开放与可复现决定团队能否持续迭代。

11.5 本章小结

一次成功的 reasoning 升级实验，不是某个单点参数的胜利，而是目标约束、阶段矩阵、错误分析和预算管理协同工作的结果。

12 总结与延伸

12.1 全讲总结表

阶段	核心问题	本讲给出的可执行答案
Pre-training	底座能力如何在预算内做强	用可复现 OLMo 配方优化数据与训练细节
Post-training	如何把能力变成可用行为与推理优势	SFT 打协议，DPO/PPO 调偏好，RLVR 冲可验证推理
Inference-time	如何在部署端进一步提效	按请求价值使用 test-time scaling 与检索校验闭环
开放生态	如何让改进可累积	开放数据/代码/评估链路，强化可复现与可审计

表 8: Open Training Recipes for Reasoning 的结构化结论

12.2 核心结论

一句话总结

Reasoning 能力是系统工程结果，不是单点技巧；最有效路径是“可复现预训练 + 分阶段后训练 + 预算化测试时扩展”的组合配方。

本讲对实践者最有价值的三点

1. 先构建可复现实验链路，再追求更高分数；
2. 用可验证奖励扩大 RL 训练可自动化范围；
3. 将 test-time scaling 作为部署预算管理能力的，而不是临时补丁。

12.3 进一步阅读

- OLMo / OLMo 2 技术报告与训练代码 (AI2)

- Tulu 3 后训练配方论文与开源实现
- DPO / PPO 系列偏好优化论文与长度偏置修正方法
- RLVR 与 DeepSeek-R1 路线相关公开资料
- Self-RAG 与 test-time scaling 相关研究

12.4 延伸问题

值得继续追踪的开放问题

- 如何为非数学非代码任务构建低成本可信验证器？
- 如何建立跨模型、跨数据版本的一致评估协议？
- 如何在固定预算下动态决定“再采样”是否值得？